BusyBox

Why BusyBox?

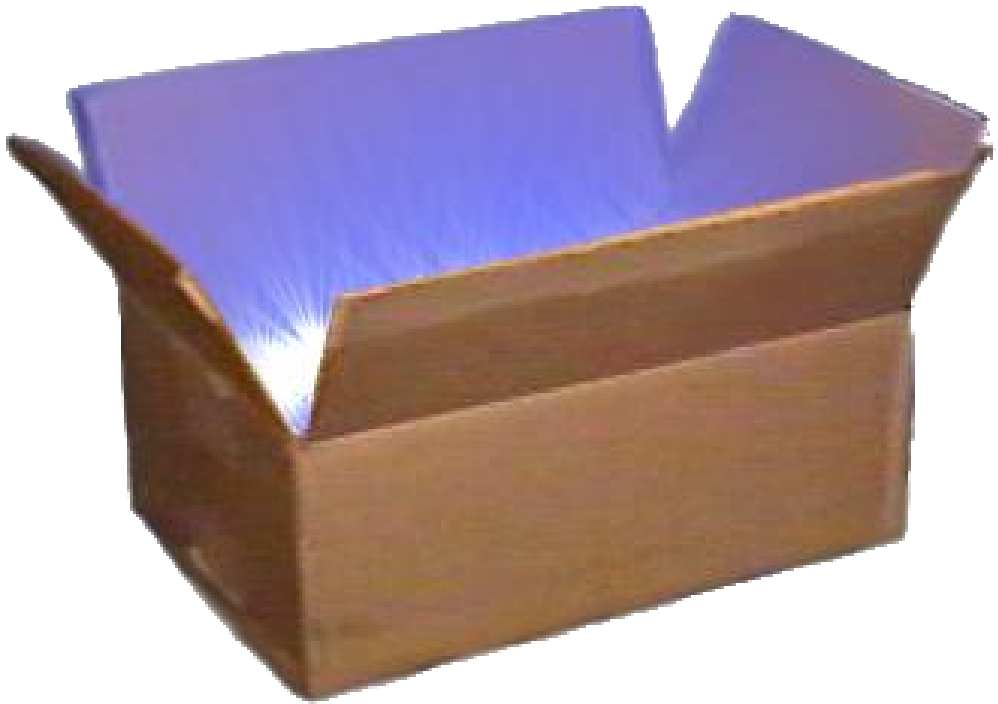
- A Linux system needs a basic set of programs to work
  - An init program
  - A shell
  - Various basic utilities for file manipulation and system configuration
- In normal GNU/Linux systems, these programs are provided by different projects
  - `coreutils`, `bash`, `grep`, `sed`, `tar`, `wget`, `modutils`, etc. are all different projects
  - A lot of different components to integrate
  - Components not designed with embedded systems constraints in mind: they are not very configurable and have a wide range of features
- BusyBox is an alternative solution, extremely common on embedded systems

General purpose toolbox: BusyBox

https://www.busybox.net/

- Rewrite of many useful UNIX command line utilities
  - Created in 1995 to implement a rescue and installer system for Debian, fitting in a single floppy disk (1.44 MB)
  - Integrated into a single project, which makes it easy to work with
  - Great for embedded systems: highly configurable, no unnecessary features
  - Called the *Swiss Army Knife of Embedded Linux*
- License: GNU GPLv2
- Alternative: Toybox, BSD licensed (https://en.wikipedia.org/wiki/Toybox)

General purpose toolbox: BusyBox



BusyBox in the root filesystem

- All the utilities are compiled into a single executable, `/bin/busybox`
  - Symbolic links to `/bin/busybox` are created for each application integrated into BusyBox
- For a fairly featureful configuration, less than 500 KB (statically compiled with uClibc) or less than 1 MB (statically compiled with glibc).

BusyBox in the root filesystem

```
rootfs
├── bin
│   ├── ash -> busybox
│   ├── busybox
│   ├── cat -> busybox
│   ├── ls -> busybox
│   ├── mount -> busybox
│   └── sh -> busybox
├── sbin
│   ├── halt -> ../bin/busybox
│   ├── ifconfig -> ../bin/busybox
│   └── init -> ../bin/busybox
└── usr
    └── sbin
        └── httpd -> ../../bin/busybox
```

BusyBox - Most commands in one binary

[, [[, acpid, add-shell, addgroup, adduser, adjtimex, arch, arp, arping, ash, awk, base64, basename, bc, beep, blkdiscard, blkid, blockdev, bootchartd, brctl, bunzip2, bzcat, bzip2, cal, cat, chat, chattr, chgrp, chmod, chown, chpasswd, chpst, chroot, chrt, chvt, cksum, clear, cmp, comm, conspy, cp, cpio, crond, crontab, cryptpw, cttyhack, cut, date, dc, dd, deallocvt, delgroup, deluser, depmod, devmem, df, dhcprelay, diff, dirname, dmesg, dnsd, dnsdomainname, dos2unix, dpkg, dpkg-deb, du, dumpkmap, dumpleases, echo, ed, egrep, eject, env, envdir, envuidgid, ether-wake, expand, expr, factor, fakeidentd, fallocate, false, fatattr, fbset, fbsplash, fdflush, fdformat, fdisk, fgconsole, fgrep, find, findfs, flock, fold, free, freeramdisk, fsck, fsck.minix, fsfreeze, fstrim, fsync, ftpd, ftpget, ftpput, fuser, getopt, getty, grep, groups, gunzip, gzip, halt, hd, hdparm, head, hexdump, hexedit, hostid, hostname, httpd, hush, hwclock, i2cdetect, i2cdump, i2cget, i2cset, i2ctransfer, id, ifconfig, ifdown, ifenslave, ifplugd, ifup, inetd, init, insmod, install, ionice, iostat, ip, ipaddr, ipcalc, ipcrm, ipcs, iplink, ipneigh, iproute, iprule, iptunnel, kbd_mode, kill, killall, killall5, klogd, last, less, link, linux32, linux64, linuxrc, ln, loadfont, loadkmap, logger, login, logname, logread, losetup, lpd, lpq, lpr, ls, lsattr, lsmod, lsof, lspci, lsscsi, lsusb, lzcat, lzma, lzop, makedevs, makemime, man, md5sum, mdev, mesg, microcom, mim, mkdir, mkdosfs, mke2fs, mkfifo, mkfs.ext2, mkfs.minix, mkfs.vfat, mknod, mkpasswd, mkswap, mktemp, modinfo, modprobe, more, mount, mountpoint, mpstat, mt, mv, nameif, nanddump, nandwrite, nbd-client, nc, netstat, nice, nl, nmeter, nohup, nologin, nproc, nsenter, nslookup, ntpd, nuke, od, openvt, partprobe, passwd, paste, patch, pgrep, pidof, ping, ping6, pipe_progress, pivot_root, pkill, pmap, popmaildir, poweroff, powertop, printenv, printf, ps, pscan, pstree, pwd, pwdx, raidautorun, rdate, rdev, readahead, readlink, readprofile, realpath, reboot, reformime, remove-shell, renice, reset, resize, resume, rev, rm, rmdir, rmmod, route, rpm, rpm2cpio, rtcwake, run-init, run-parts, runlevel, runsv, runsvdir, rx, script, scriptreplay, sed, sendmail, seq, setarch, setconsole, setfattr, setfont, setkeycodes, setlogcons, setpriv, setserial, setsid, setuidgid, sh, sha1sum, sha256sum, sha3sum, sha512sum, showkey, shred, shuf, slattach, sleep, smemcap, softlimit, sort, split, ssl_client, start-stop-daemon, stat, strings, stty, su, sulogin, sum, sv, svc, svlogd, svok, swapoff, swapon, switch_root, sync, sysctl, syslogd, tac, tail, tar, taskset, tc, tcpsvd, tee, telnet,

BusyBox - Most commands in one binary

```
telnetd, test, tftp, tftpd, time, timeout, top, touch, tr, traceroute, traceroute6, true, truncate, ts, tty,
ttysize, tunctl, ubiattach, ubidetach, ubimkvol, ubirename, ubirmvol, ubirsvol, ubiupdatevol, udhcpc, udhcpc6,
udhcpd, udpsvd, uevent, umount, uname, unexpand, uniq, unix2dos, unlink, unlzma, unshare, unxz, unzip, uptime,
users, usleep, uudecode, uuencode, vconfig, vi, vlock, volname, w, wall, watch, watchdog, wc, wget, which, who,
whoami, whois, xargs, xxd, xz, xzcat, yes, zcat, zcip
```

Source: run `/bin/busybox` - July 2021 status

Configuring BusyBox

- Get the latest stable sources from `https://busybox.net`
- Configure BusyBox (creates a `.config` file):
  - `make defconfig` Good to begin with BusyBox. Configures BusyBox with all options for regular users.
  - `make allnoconfig` Unselects all options. Good to configure only what you need.
- `make menuconfig` (text) Same configuration interfaces as the ones used by the Linux kernel (though older versions are used, causing `make xconfig` to be broken in recent distros).

BusyBox make menuconfig

You can choose:

- the commands to compile,
- and even the command options and features that you need!

```
                                Coreutils
Arrow keys navigate the menu.  <Enter> selects submenus --->.  Highlighted
letters are hotkeys.  Pressing <Y> includes, <N> excludes, <M> modularizes
features.  Press <Esc><Esc> to exit, <?> for Help, </> for Search.  Legend:
[*] built-in  [ ] excluded  <M> module  < > module capable
    ↑(-)
    [ ] link (3.2 kb)
    [*] ln (4.9 kb)
    [ ] logname (1.1 kb)
    [*] ls (14 kb)
    [*]   Enable filetyping options (-p and -F)
    [ ]   Enable symlinks dereferencing (-L)
    [*]   Enable recursion (-R)
    [*]   Enable -w WIDTH and window size autodetection
    [*]   Sort the file names
    [*]   Show file timestamps
    [*]   Show username/groupnames
    [ ]   Allow use of color to identify file types
    [*] md5sum (6.5 kb)
    ↓(+)

                    <Select>    < Exit >    < Help >
```

Compiling BusyBox

- Set the cross-compiler prefix in the configuration interface: `Settings -> Build Options -> Cross Compiler prefix` Example: `arm-linux-`
- Set the installation directory in the configuration interface: `Settings -> Installation Options ‘ -> Destination path for make install'`
- Add the cross-compiler path to the PATH environment variable: `export PATH=\$H O M E slash x minus t o o l s slash a r m minus u n k n o w n minus l i n u x minus u c l i b c g n u e a b i slash b i n colon\$PATH`
- Compile BusyBox: `make`
- Install it (this creates a UNIX directory structure with symbolic links to the `busybox` executable): `make install`

Applet highlight: BusyBox init

- BusyBox provides an implementation of an `init` program
- Simpler than the init implementation found on desktop/server systems (*SysV init* or *systemd*)
- A single configuration file: `/etc/inittab`
  - Each line has the form `<id>::<action>:<process>`
- Allows to start system services at startup, to control system shutdown, and to make sure that certain services are always running on the system.
- See examples/inittab in BusyBox for details on the configuration

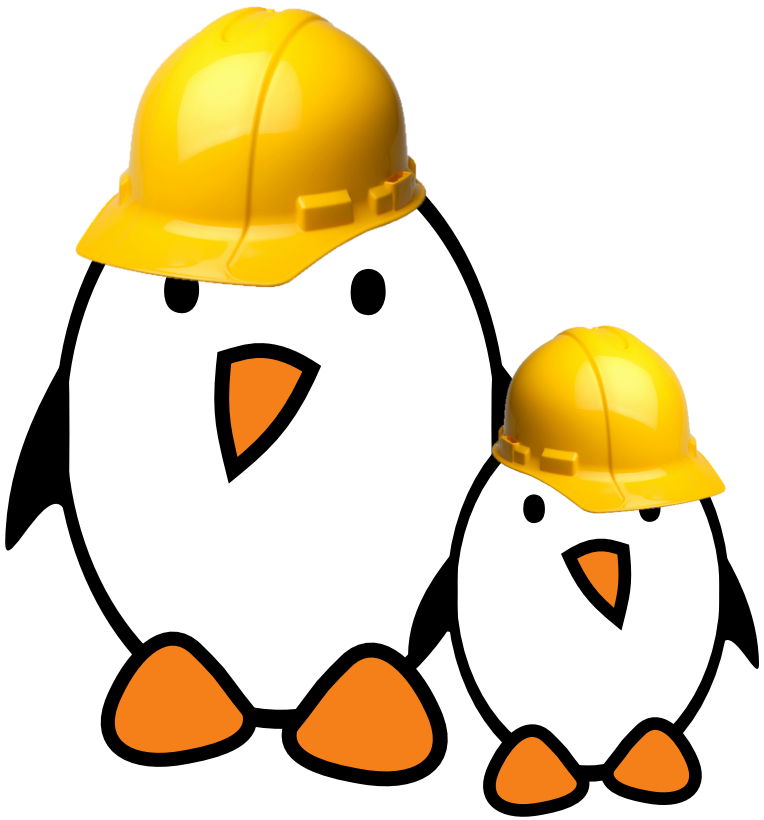
Applet highlight: BusyBox vi

- If you are using BusyBox, adding `vi` support only adds about 20K
- You can select which exact features to compile in.
- Users hardly realize that they are using a lightweight `vi` version!
- Tip: you can learn `vi` on the desktop, by running the `vimtutor` command.

Applet highlight: BusyBox vi

```
[ ] cmp (4.9 kb)
[ ] diff (13 kb)
[ ] ed (21 kb)
[ ] patch (9.4 kb)
[ ] sed (12 kb)
[*] vi (23 kb)
(4096) Maximum screen width
[*]   Allow to display 8-bit chars (otherwise shows dots)
[*]   Enable ":" colon commands (no "ex" mode)
[*]   Enable yank/put commands and mark cmds
[*]   Enable search and replace cmds
[ ]     Enable regex in search and replace
[*]   Catch signals
[*]   Remember previous cmd and "." cmd
[*]   Enable -R option and "view" mode
[*]   Enable settable options, ai ic showmatch
[*]   Support :set
[ ]   Handle window resize
[ ]   Use 'tell me cursor position' ESC sequence to measure window
[*]   Support undo command "u"
[*]     Enable undo operation queuing
(256)     Maximum undo character queue size
[ ]   Allow vi and awk to execute shell commands
```

Practical lab - Tiny root filesystem built from scratch with BusyBox



)

- Setting up a kernel to boot your system on a workstation directory exported by NFS
- Passing kernel command line parameters to boot on NFS
- Creating the full root filesystem from scratch. Populating it with BusyBox based utilities.
- System startup using BusyBox `init`
- Using the BusyBox HTTP server.
- Controlling the target from a web browser on the PC host.
- Setting up shared libraries on the target and compiling a sample executable.